

Name:Manav Mangesh Uttekar

Roll no.: SE-71

Problem statement: Consider telephone book database of N clients. Make use of a hash table implementation to quickly look up client's telephone number.Make use of two collision handling techniques and compare them using number of comparisons required to find a set of telephone numbers

```
def simple_hash(key, size):
    return sum(ord(c) for c in key) % size

class HashTableChaining:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(size)]
        self.comparisons = 0

    def insert(self, key, value):
        index = simple_hash(key, self.size)
        for i in range(len(self.table[index])):
            self.comparisons += 1
            if self.table[index][i][0] == key:
                self.table[index][i] = (key, value)
                return
        self.table[index].append((key, value))

    def search(self, key):
        index = simple_hash(key, self.size)
        for k, v in self.table[index]:
            self.comparisons += 1
            if k == key:
                return v
        return None

class HashTableLinearProbing:
    def __init__(self, size):
        self.size = size
        self.table = [None] * size
        self.comparisons = 0

    def insert(self, key, value):
        index = simple_hash(key, self.size)
        start_index = index
        while self.table[index] is not None:
            self.comparisons += 1
            if self.table[index][0] == key:
                break
            index = (index + 1) % self.size
            if index == start_index:
                print("Error: Hash table full")
                return
        self.table[index] = (key, value)

    def search(self, key):
        index = simple_hash(key, self.size)
```

```

        start_index = index
        while self.table[index] is not None:
            self.comparisons += 1
            if self.table[index][0] == key:
                return self.table[index][1]
            index = (index + 1) % self.size
            if index == start_index:
                break
        return None

def main():
    print("=== Telephone Book using Hash Tables ===")
    n = int(input("Enter number of clients: "))
    chaining = HashTableChaining(size=n)
    probing = HashTableLinearProbing(size=2 * n)

    for i in range(n):
        name = input(f"Enter name of client {i+1}: ")
        number = input(f"Enter telephone number of {name}: ")
        chaining.insert(name, number)
        probing.insert(name, number)

    m = int(input("\nEnter number of clients to search: "))
    to_search = []
    for i in range(m):
        name = input(f"Enter name to search ({i+1}): ")
        to_search.append(name)

    print("\n=== Search Results ===")
    for name in to_search:
        ch_result = chaining.search(name)
        lp_result = probing.search(name)

        print(f"{name}:")
        if ch_result is not None:
            print(f"    Chaining Result: {ch_result}")
        else:
            print(f"    Chaining Result: Not found")

        if lp_result is not None:
            print(f"    Linear Probing Result: {lp_result}")
        else:
            print(f"    Linear Probing Result: Not found")

    print("\n=== Comparisons Count ===")
    print("Chaining comparisons:", chaining.comparisons)
    print("Linear Probing comparisons:", probing.comparisons)

if __name__ == "__main__":
    main()

```

OUTPUT:

```
PS C:\Users\Manav\Desktop\exp of DSA> python -u "c:\Users\Manav\Desktop\exp of DSA\exp1.py"
=== Telephone Book using Hash Tables ===
Enter number of clients: 2
Enter name of client 1: rohit
Enter telephone number of rohit: 3425345345
Enter name of client 2: rayan
Enter telephone number of rayan: 3242342354

Enter number of clients to search: 1
Enter name to search (1): rayan

=== Search Results ===
rayan:
    Chaining Result: 3242342354
    Linear Probing Result: 3242342354

=== Comparisons Count ===
Chaining comparisons: 1
Linear Probing comparisons: 1
```